



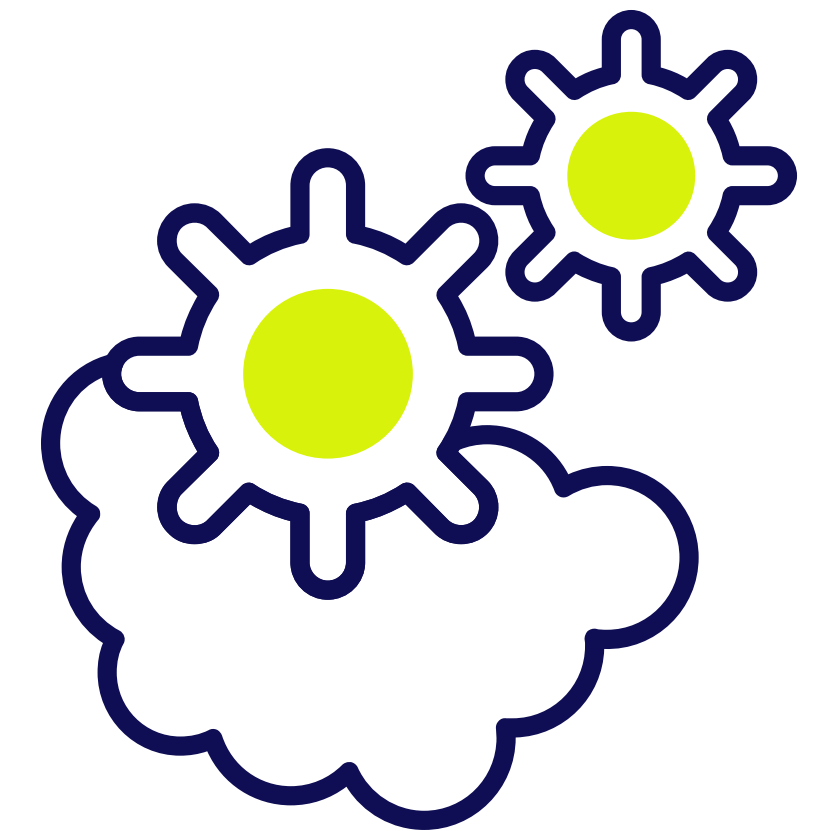
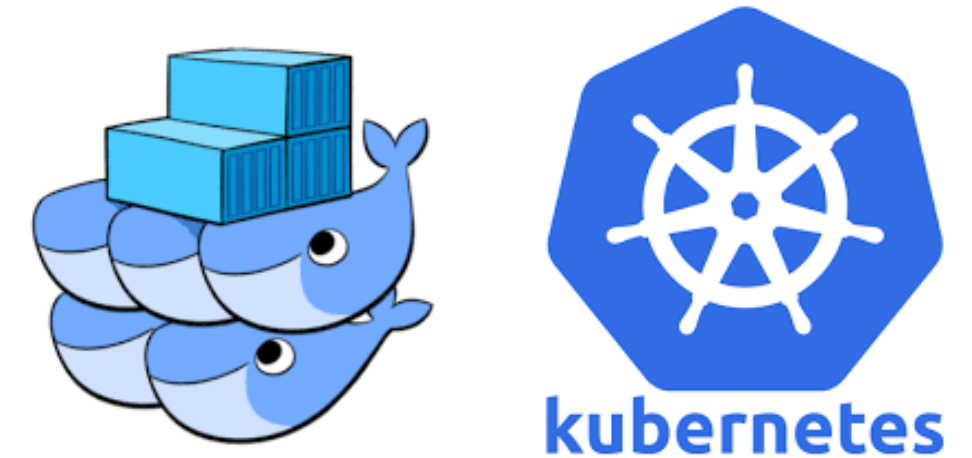
DR B. DIOP

babacar.diop@ussein.edu.sn

Conteneurisation & orchestration

Kubernetes Orchestration

Services Discovery



INFO – AGROTIC

Last update: dec. 2025

Découverte de Services

Objectif : Comprendre la découverte de services, son importance et son implémentation dans Kubernetes

- Prérequis : Connaissance des Services Kubernetes et de leur fonctionnement
- Sections :
 - Contexte rapide
 - Enregistrement des services
 - Découverte des services
 - Découverte de services et Namespaces
 - Dépannage de la découverte de services

Note : "Service" avec une majuscule fait référence à la ressource Kubernetes, "service" en minuscule au concept général.

Le Problème et la Solution

- **Problème** : Dans Kubernetes, les applications (Pods) sont éphémères, scalent, s'auto-soignent → adresses IP instables
- **Solution** : L'objet Service Kubernetes fournit un point d'entrée stable (IP virtuelle, nom DNS, port)
- **Défi** : Comment les applications se trouvent-elles entre elles ?
- **Réponse** : La découverte de services, composée de :
 - Enregistrement (Service Registration)
 - Découverte (Service Discovery)

Architecture de la Découverte de Services

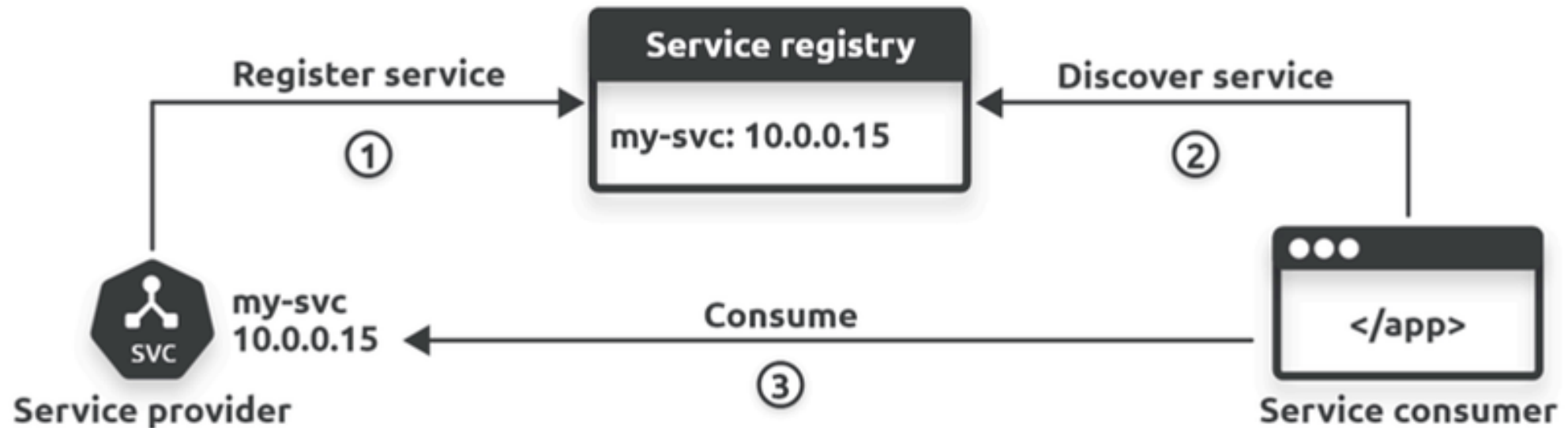
Vue d'Ensemble :

- **Service Registry** (Registre) : DNS interne du cluster (CoreDNS)
- **Enregistrement Automatique** : Tous les **Services** s'enregistrent automatiquement auprès du **DNS**
- Composants Clés :
 - **kube-dns/coredns** (**Pods** + **Service** dans le Namespace **kube-system**)
 - **kube-proxy** (sur chaque nœud)
 - **Endpoints/EndpointSlice** (liste des **Pods** sains)
 - **Règles IPVS/iptables** (routage local)

Architecture de la Découverte de Services

Illustration :

- **Service Registry** (Registre) : DNS interne du cluster (CoreDNS)
- **Enregistrement Automatique** : Tous les **Services** s'enregistrent automatiquement auprès du **DNS**



Enregistrement des Services - Processus

Flux d'Enregistrement

1. Déploiement d'un manifeste Service via **kubectl apply**
2. Authentification/autorisation par l'**API Server**
3. Attribution d'une **ClusterIP** (IP virtuelle stable)
4. Création d'un objet **Endpoints/EndpointSlice** avec les **IPs** des **Pods** correspondant au sélecteur (**label selector**)
5. Configuration du réseau pour gérer le trafic vers la **ClusterIP**
6. Enregistrement **DNS** automatique : Le contrôleur **CoreDNS** observe les nouveaux **Services** et crée les enregistrements A/SRV.

Résultat :

Le Service est joignable via **<nom_du_service>.<namespace>.svc.cluster.local**

Backend des Services - Endpoints

Comment le Trafic Atteint les Pods ?

- Chaque **Service** a un **label selector** qui détermine les **Pods** cibles
- **Endpoints/EndpointSlice** maintient la liste des **IP:port** des **Pods** sains
- **kube-proxy** (sur chaque nœud) :
 - Observe les **Services** et Endpoints via l'**API Server**
 - Configure **IPVS** (ou iptables) pour rediriger le trafic **ClusterIP** → **Pods**
- **IPVS : Load-balancer** couche 4 performant dans le noyau Linux (remplace iptables depuis K8s version 1.11)



Lister des Endpoints d'un Service

Exemple

Lister les **Endpoint** d'un **Service** appelé **ent** :

```
$ kubectl get endpoint ent
```

NAME	ENDPOINTS	AGE
ent	192.168.129.46:8080,192.168.130.127:8080	14m

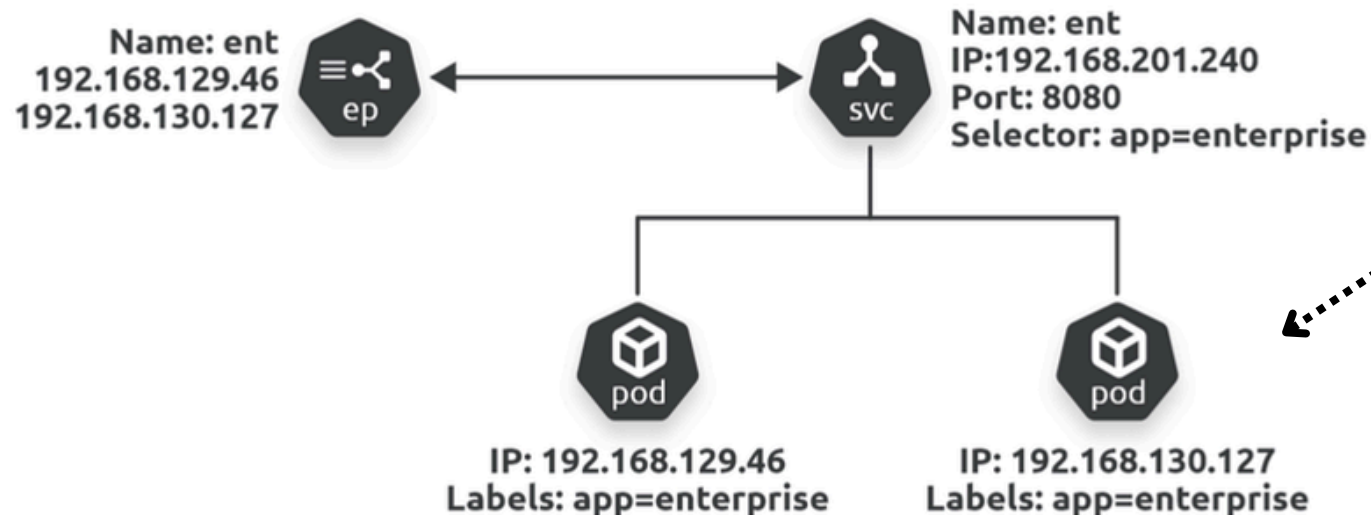
Lister des Endpoints d'un Service

Exemple

Lister les **Endpoint** d'un **Service** appelé **ent** :

```
$ kubectl get endpoint ent
```

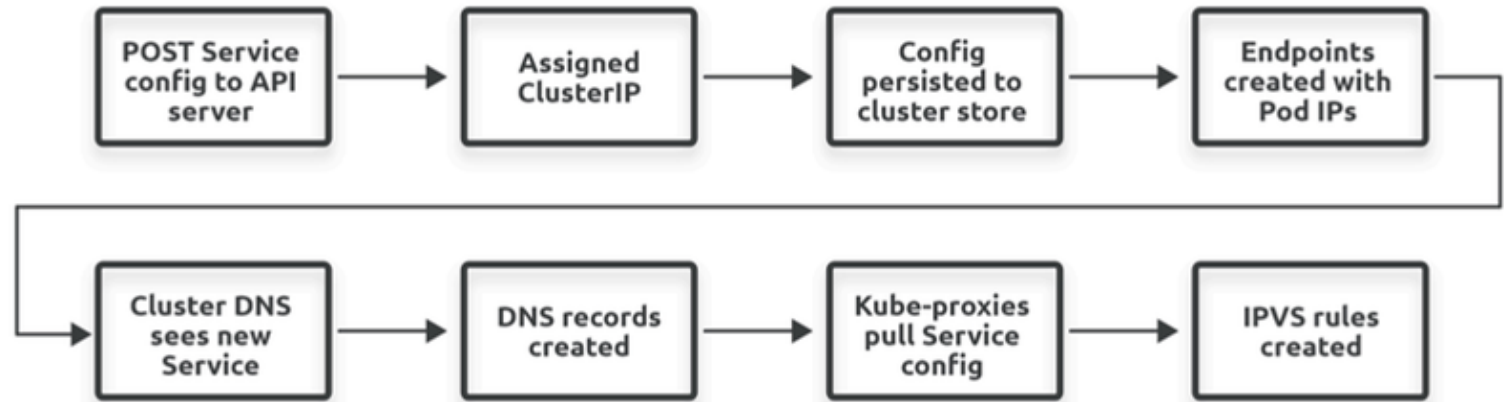
NAME	ENDPOINTS	AGE
ent	192.168.129.46:8080,192.168.130.127:8080	14m



Same ent **Service** that will load-balance traffic to two **Pods**, with the **Endpoints** object with the **IPs** of the two **Pods** matching the Service's label selector

Schéma complet régistration

1. Publier un nouveau manifeste de ressource de service sur le **serveur API** où il est authentifié et autorisé.
2. Le service se voit attribuer une **adresse IP de cluster** et sa configuration est enregistrée dans le stockage du cluster.
3. Un objet **Endpoints** ou **EndpointSlice** associé est créé pour contenir la liste des adresses IP des **Pods** sains correspondant au sélecteur d'étiquettes.

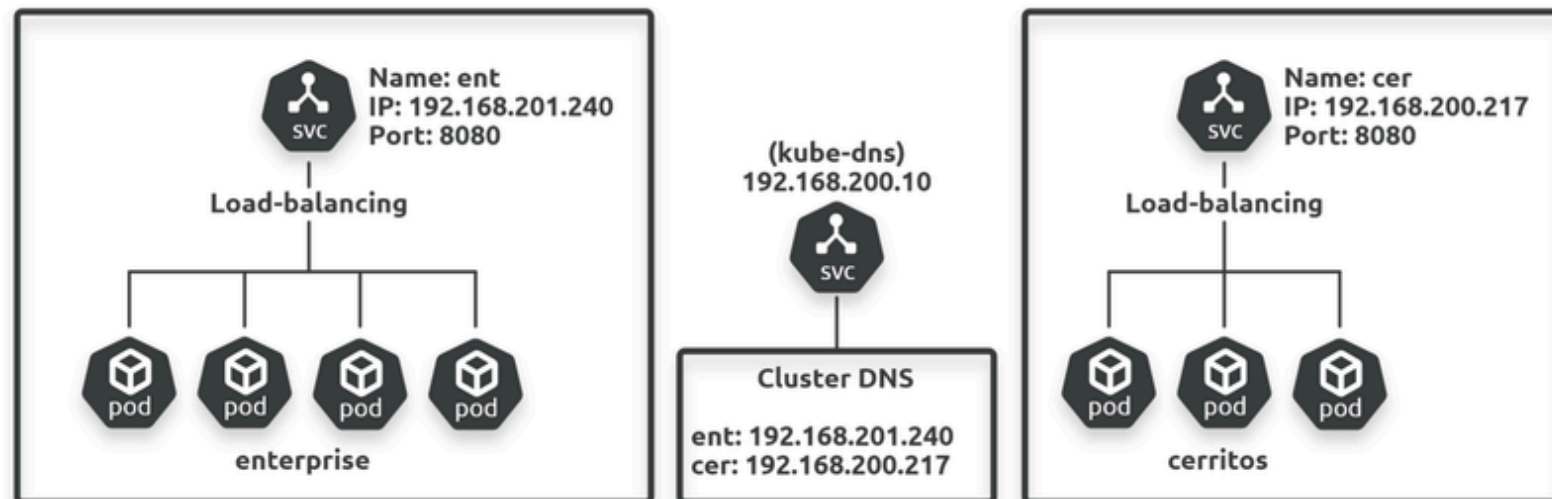


4. Le **cluster DNS** s'exécute comme une application native **Kubernetes** et surveille le **APIServer** à la recherche de nouveaux objets de service. Il les observe et enregistre les **enregistrements DNS A** et **SRV** appropriés.
5. Chaque nœud exécute un **kube-proxy** qui observe les **nouveaux objets** et crée des règles **IPVS/iptables** locales afin que le trafic destiné à l'adresse IP de cluster du service soit acheminé vers les **Pods** correspondant au sélecteur d'étiquettes du service.

Service discovery problem

Supposons qu'il existe **deux applications de microservices** sur le **même cluster** Kubernetes : **Enterprise** et **Cerritos**. Les pods d'**Enterprise** sont hébergés par un service nommé **ent**, et ceux de **Cerritos** par un autre service nommé **cer**. Ils possèdent des adresses IP de cluster (**ClusterIP**) et sont enregistrés auprès du **DNS**. Voici leur configuration.

App	Service name	ClusterIP
Enterprise	ent	192.168.201.240
Cerritos	cer	192.168.200.217



Service discovery problem

Pour que la découverte de services fonctionne, les applications doivent connaître les deux éléments suivants :

1. **Service frontal** : le nom des autres applications auxquelles elles souhaitent se connecter.
2. **Service frontal -->> @IP** : Comment convertir ce nom en adresse IP.

Les développeurs d'applications sont responsables de coder les noms des services frontaux des applications distantes. Par exemple, si l'application **Cerritos** souhaite se connecter à **Enterprise**, elle doit connaître le nom du **Service** « ent ».

Kubernetes se charge du point 2, en convertissant le nom en adresse IP.

Découverte de Services - Processus

Comment une Application Trouve un Service ?

Exemple : L'application "entreprise" veut joindre "cerritos"

1. **Connaissance du nom** : Le développeur code **cer** (nom du Service cible)

2. Résolution DNS :

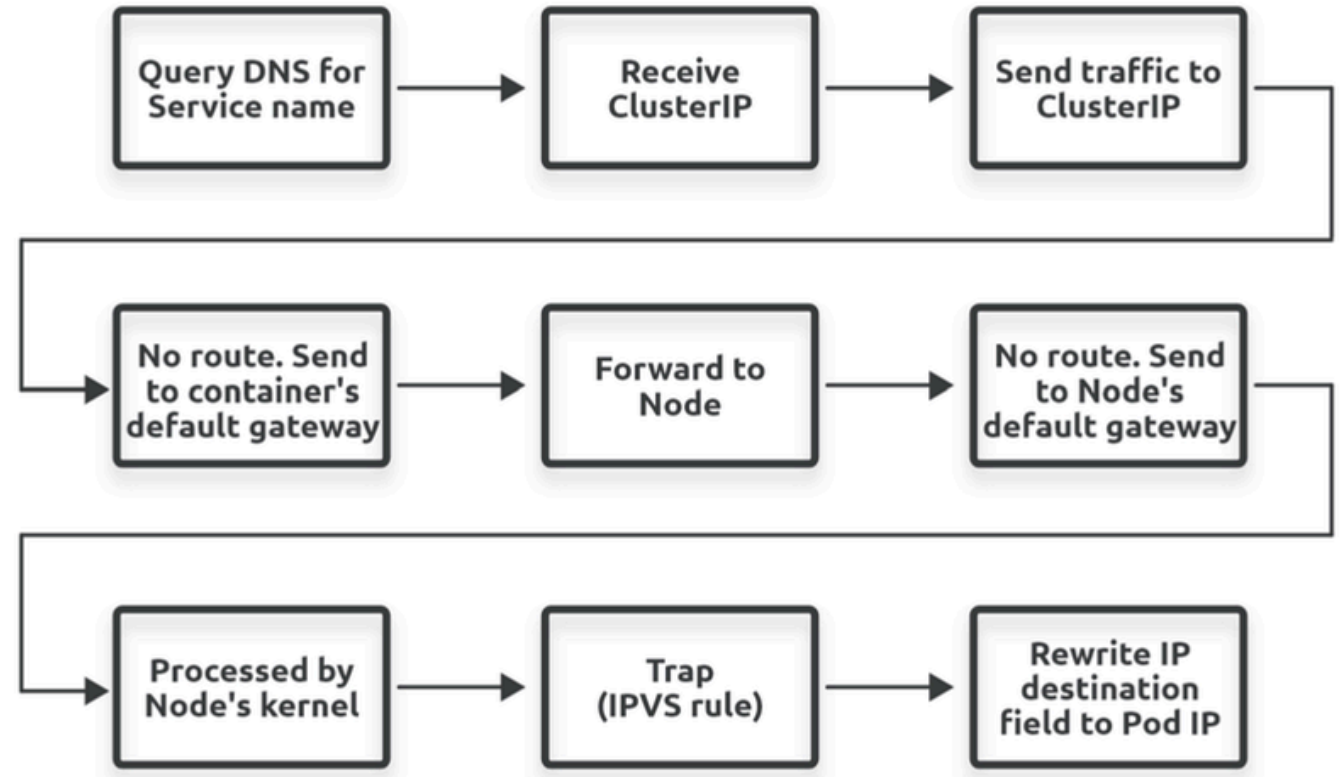
- Le container envoie la requête à **CoreDNS** (via **/etc/resolv.conf**)
- Réponse : **ClusterIP** du Service **cer**

3. Routage Réseau :

- Trafic envoyé à la **ClusterIP** (réseau virtuel de services)
- Pas de route directe → passe par la passerelle par défaut
- **Interception par IPVS** sur le nœud : redirection vers un Pod cible

Découverte de Services - Processus

1. Une instance de l'app "entreprise" essaie d'envoyer du trafic au Service "cer". Le conteneur hébergeant l'app "entreprise" envoie le nom "cer" au cluster DNS pour le résoudre en adresse IP. Le conteneur sait comment faire car il est pré-configuré avec l'adresse du DNS du cluster dans son fichier `/etc/resolv.conf`.

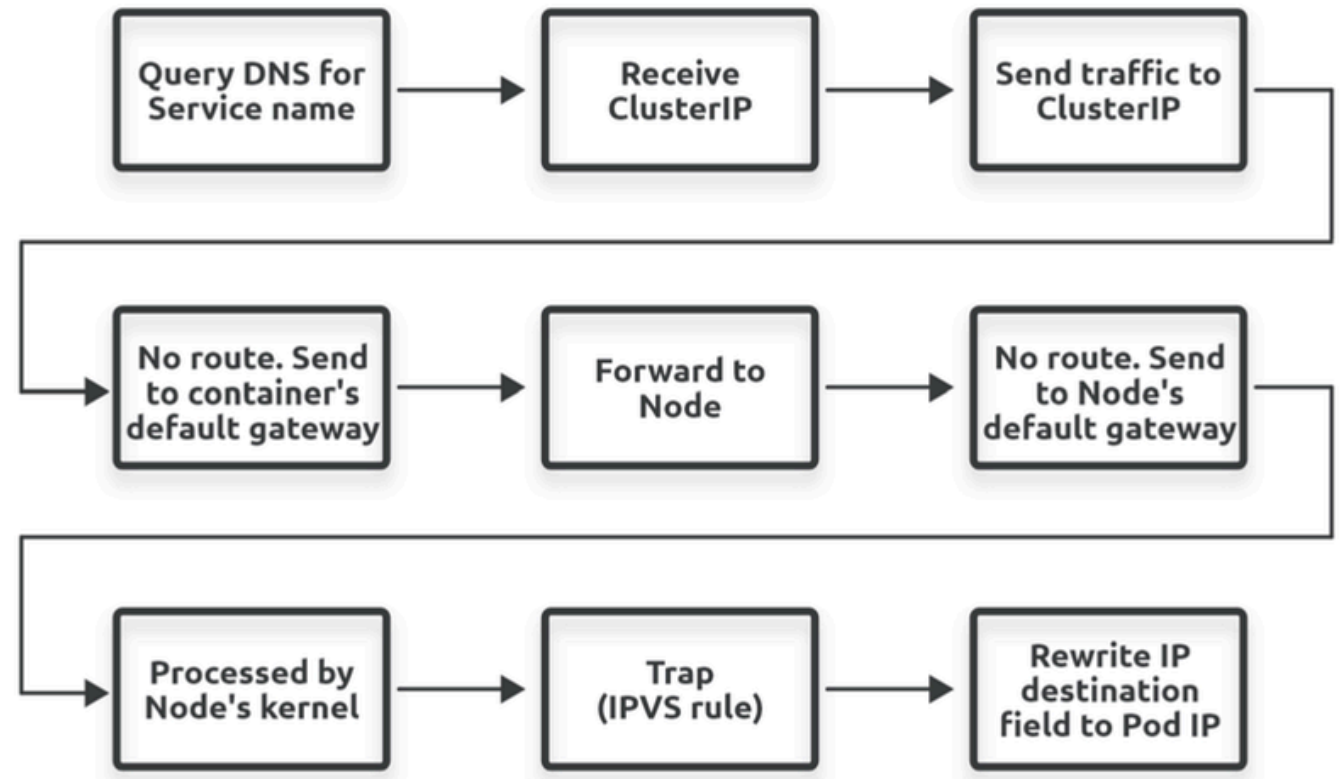


Découverte de Services - Processus

2. Le DNS du cluster répond avec la ClusterIP et le conteneur "entreprise" envoie le trafic vers le réseau.

3. Cependant, les ClusterIP sont sur le réseau de service spécial et le conteneur n'a pas de route vers celui-ci. Donc il l'envoie à sa passerelle par défaut, qui le transmet au nœud sur lequel il s'exécute.

4. Le nœud n'a pas de route non plus, donc il l'envoie à sa propre passerelle par défaut. En chemin, la requête est traitée par le noyau du nœud.



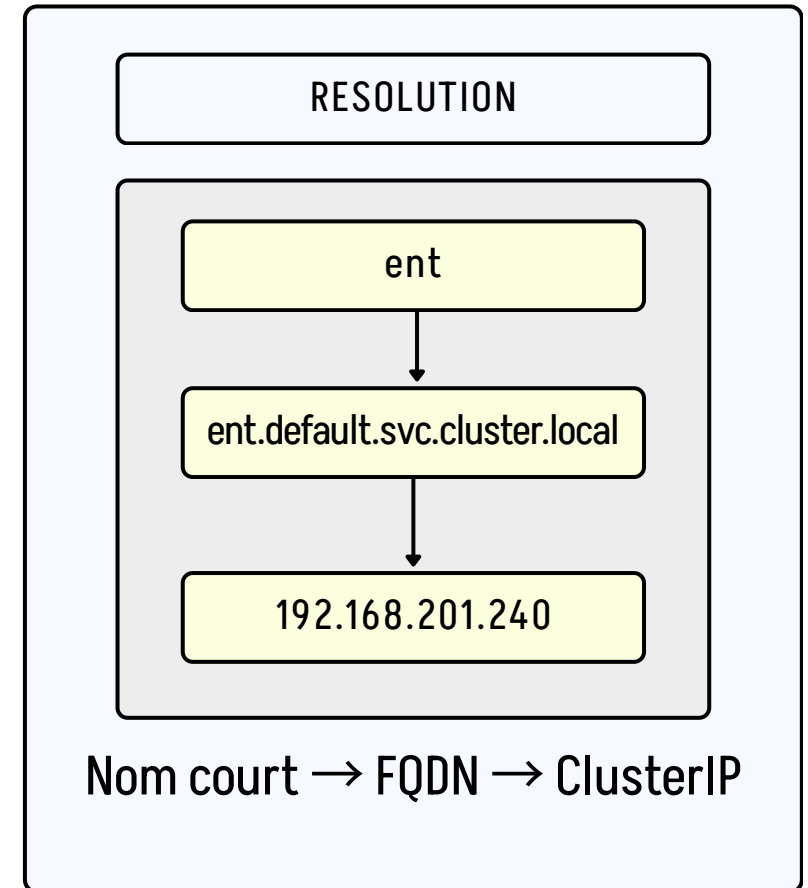
5. Un piège est déclenché, et la requête est redirigée vers l'adresse IP d'un Pod qui correspond au sélecteur de labels du Service.

Configuration DNS des Containers

/etc/resolv.conf - La clé de la résolution

```
# Exemple dans un container
$ cat /etc/resolv.conf
search default.svc.cluster.local svc.cluster.local cluster.local
nameserver 192.168.200.10 # IP du Service kube-dns
options ndots:5
```

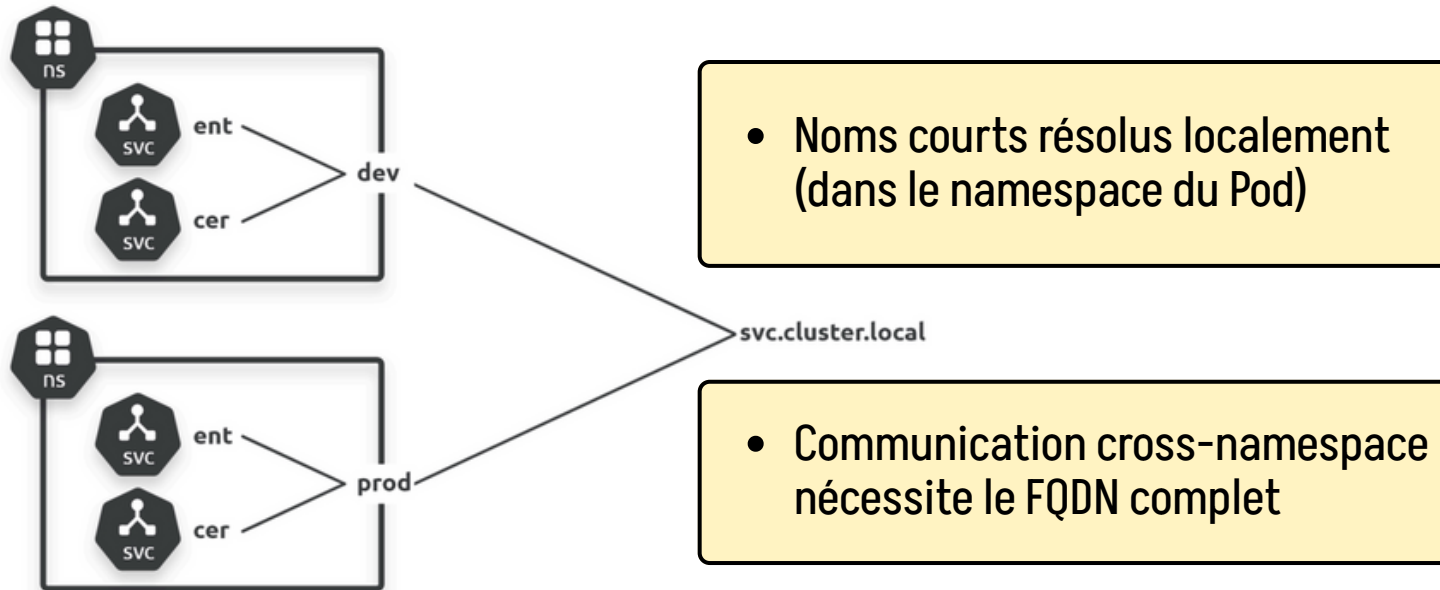
- **Search domains** : Domaines à essayer pour les noms courts
- **Nameserver** : IP de CoreDNS (identique pour tous les containers)
- **ndots:5** : Seuil pour utiliser les search domains vs FQDN



Découverte et Namespaces

Espaces de Noms et Domaines DNS

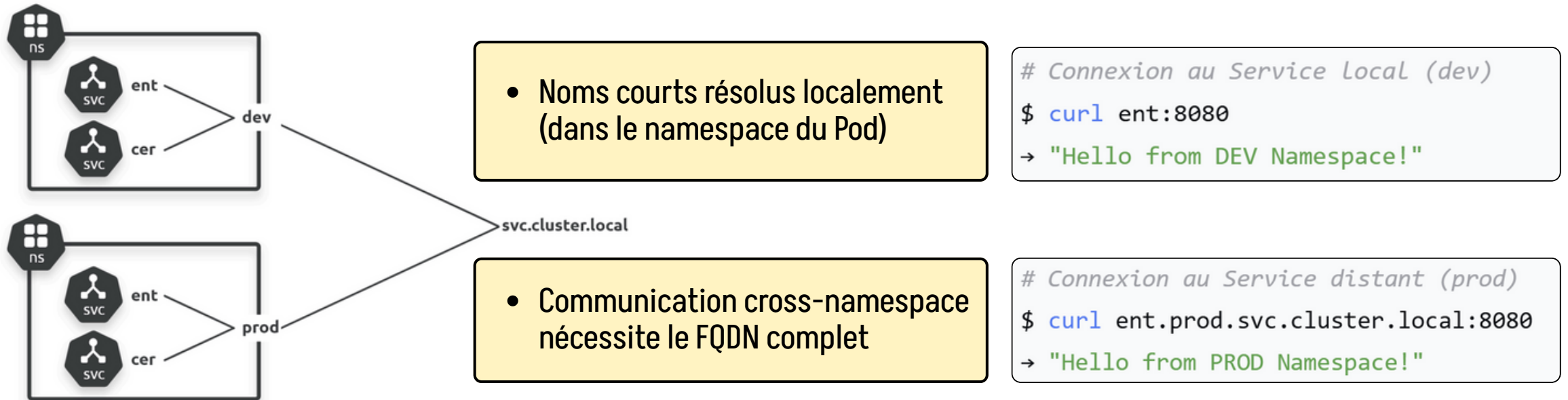
- Domaine du cluster : **cluster.local** (par défaut)
- Structure DNS : **<service>.<namespace>.svc.cluster.local**
 - Dans un même namespace : ent:8080 → ent.default.svc.cluster.local:8080
 - Dans des Namespace différent : ent.prod.svc.cluster.local:8080



Découverte et Namespaces

Espaces de Noms et Domaines DNS

- Domaine du cluster : **cluster.local** (par défaut)
- Structure DNS : **<service>.<namespace>.svc.cluster.local**
 - Dans un même namespace : ent:8080 → ent.default.svc.cluster.local:8080
 - Dans des Namespace différent : ent.prod.svc.cluster.local:8080



Dépannage - Vérification DNS

1. Vérifier CoreDNS :

```
kubectl get deploy -n kube-system -l k8s-app=kube-dns  
kubectl get pods -n kube-system -l k8s-app=kube-dns  
kubectl logs <coredns-pod> -n kube-system
```

2. Vérifier le Service kube-dns :

```
kubectl get svc kube-dns -n kube-system  
# ClusterIP doit matcher /etc/resolv.conf des containers
```

3. Vérifier les Endpoints :

```
kubectl get ep kube-dns -n kube-system  
# Doit lister les IPs des Pods CoreDNS
```

Dépannage - Tests DNS

Créer un Pod de test :

```
kubectl run dnsutils --image=gcr.io/kubernetes-e2e-test-images/dnsutils:1.3
```

Tests depuis le Pod :

```
# Résolution DNS de base
nslookup kubernetes
# Devrait retourner : kubernetes.default.svc.cluster.local

# Tester un Service spécifique
nslookup <nom-du-service>

# Vérifier la configuration
cat /etc/resolv.conf
```

Dépannage - Tests DNSérification DNS

Créer un Pod de test :

```
kubectl run dnsutils --image=gcr.io/kubernetes-e2e-test-images/dnsutils:1.3
```

Redémarrer les Pods :

```
kubectl delete pod -n kube-system -l k8s-app=kube-dns
```

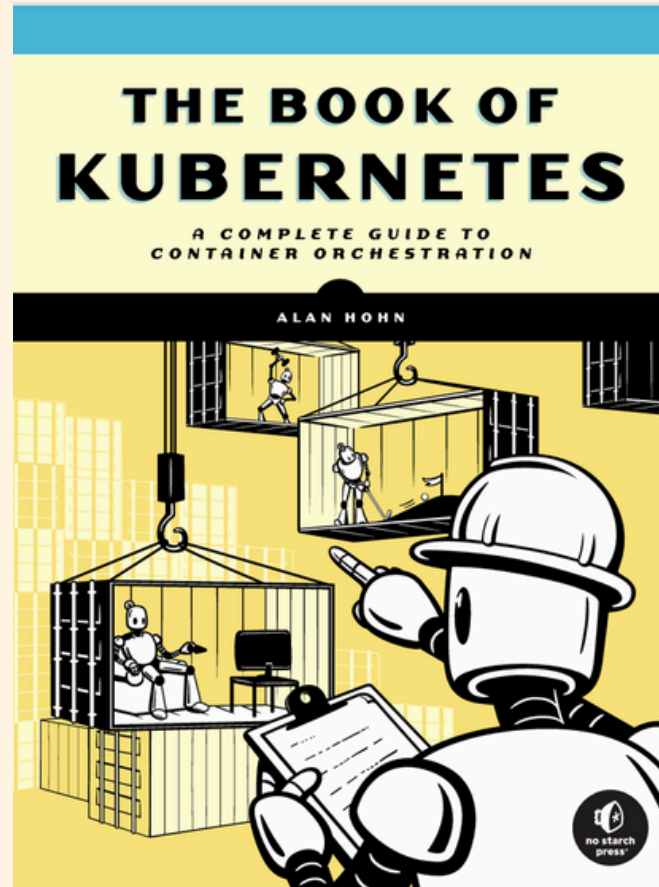
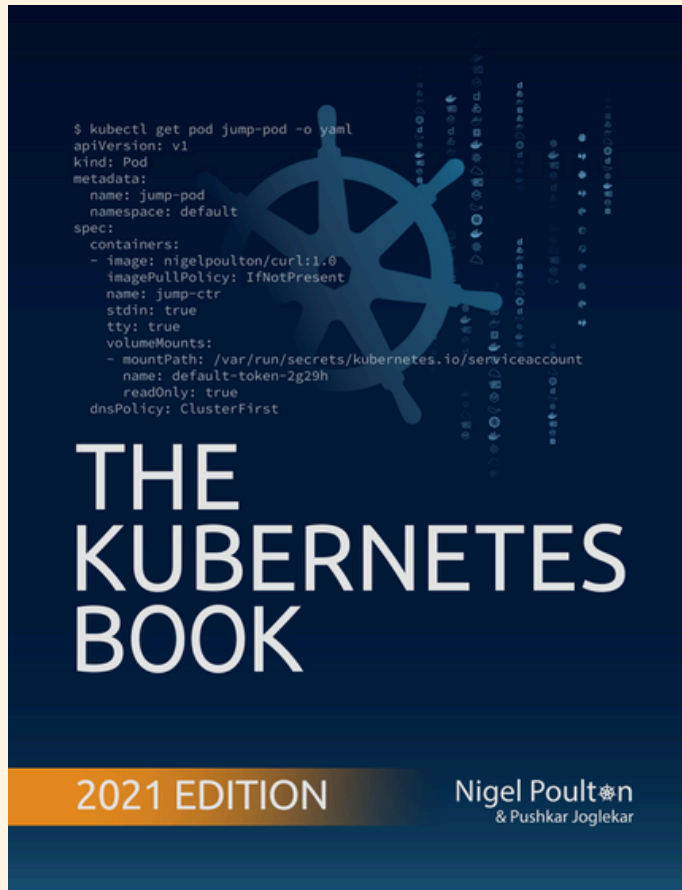
Les Pods seront recréés automatiquement

Clés à Retenir

Résumé des Concepts Essentiels

1. **DNS** = Registre : Kubernetes utilise CoreDNS comme registre de services
2. **Enregistrement Automatique** : Pas besoin d'inscrire manuellement les Services
3. **ClusterIP** : IP virtuelle stable, routée via IPVS/iptables
4. **kube-proxy** : Configure le routage local sur chaque nœud
5. **Résolution DNS** : Configurée automatiquement dans tous les containers
6. **Namespaces** : Partitionnent l'espace d'adressage DNS
7. **Dépannage** : Vérifier CoreDNS, kube-proxy, et les règles de routage

Documentation



Kubernetes Official Website Documentation

Kubernetes Documentation

Kubernetes is an open source container orchestration engine for automating deployment, scaling, and management of containerized applications. The open source project is hosted by the Cloud Native...

 [Kubernetes](#)

